Kia ora,

I'm Jason Kim, a Senior Engineer with 15+ years of experience, based in Auckland. I enjoy working on problems that aren't fully defined yet — turning rough ideas into working systems.

Recently at EmergencyQ, I worked as a sole senior engineer to deliver an Australia service expansion end-to-end — from architecture and cloud setup to backend and frontend delivery.

Previously at Visa, I worked on microservices that integrate with global banking systems, focusing on building reliable, high-availability services and distributed integrations at scale.

Across my recent work, I've focused on:
* Taking ownership of ambiguous problems and driving them through to production
* Designing pragmatic architectures across .NET, Node.js, and React
* Building and operating systems in AWS/Azure with CI/CD and serverless patterns
* Working across the stack rather than being limited to a single layer

One example that aligns strongly with this role is my work at Ambit AI:
* Led development of an AI-powered conversational system using a retrieval-based approach (early RAG pattern)
* Designed how relevant knowledge is retrieved and incorporated into prompts for grounded responses
* Delivered the solution into production as a core customer-facing capability

I'm not tied to specific tools — I focus on choosing the right approach for the problem, and I'm comfortable moving from design discussions to hands-on implementation quickly.

This role stood out to me because of its focus on building real AI-enabled systems and delivering practical outcomes — which aligns closely with how I like to work.

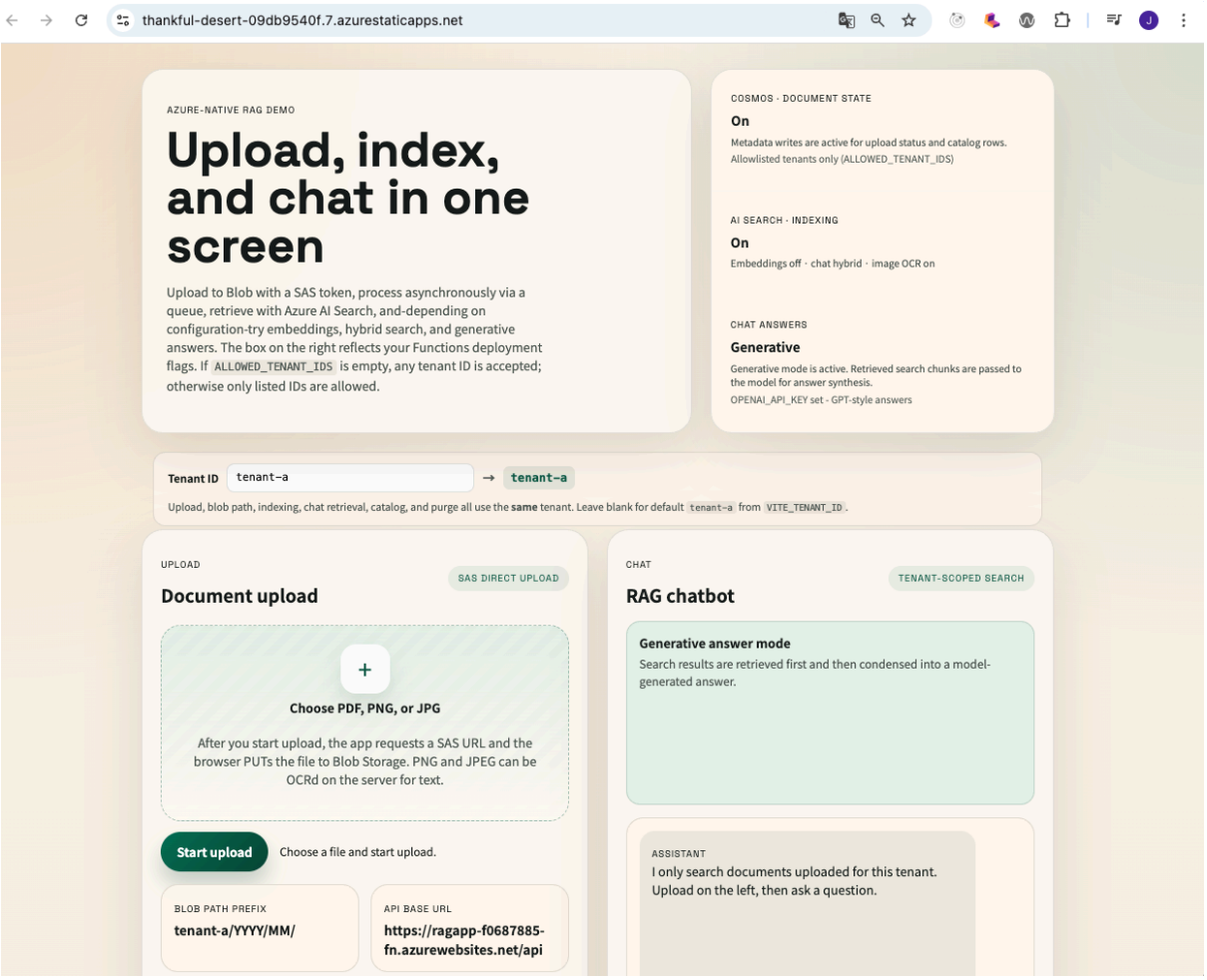
Ngā mihi,Jason Kim

==========

The following project showcases my experience across frontend, backend, CI/CD and applied AI development:

RAG (Retrieval-Augmented Generation) Chatbot System

User Manual / Portfolio
https://github.com/kimgyver/RAG-azure/blob/main/docs/user-manual-portfolio.md


thankful-desert-09db9540f.7.azurestaticapps.net
AZURE-NATIVE RAG DEMO
Upload, index, and chat in one screen
Upload to Blob with a SAS token, process asynchronously via a queue, retrieve with Azure AI Search, and-depending on configuration-try embeddings, hybrid search, and generative answers. The box on the right reflects your Functions deployment flags. If ALLOWED_TENANT_IDS is empty, any tenant ID is accepted; otherwise only listed IDs are allowed.
COSMOS · DOCUMENT STATE
On
Metadata writes are active for upload status and catalog rows.
Allowlisted tenants only (ALLOWED_TENANT_IDS)
AI SEARCH · INDEXING
On
Embeddings off · chat hybrid · image OCR on
CHAT ANSWERS
Generative
Generative mode is active. Retrieved search chunks are passed to the model for answer synthesis.
OPENAI_API_KEY set · GPT-style answers
Tenant ID
tenant-a
tenant-a
Upload, blob path, indexing, chat retrieval, catalog, and purge all use the same tenant. Leave blank for default tenant-a from VITE_TENANT_ID.
UPLOAD
SAS DIRECT UPLOAD
Document upload
Choose PDF, PNG, or JPG
After you start upload, the app requests a SAS URL and the browser PUTs the file to Blob Storage. PNG and JPEG can be OCRd on the server for text.
Start upload
Choose a file and start upload.
BLOB PATH PREFIX
tenant-a/YYYY/MM/
API BASE URL
https://ragapp-f0687885-fn.azurewebsites.net/api
CHAT
TENANT-SCOPED SEARCH
RAG chatbot
Generative answer mode
Search results are retrieved first and then condensed into a model-generated answer.
ASSISTANT
I only search documents uploaded for this tenant. Upload on the left, then ask a question.